

Security Review Report for Tomorrow Protocol

September 2026



Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
 - Security Review Lead
 - Scope
 - Changelog
4. Severity Structure
 - Severity characteristics
 - Issue symbolic codes
5. Findings Summary
6. Weaknesses
 - Replacing the fee controller resets the high-water mark
 - Management fees accrue before the first deposit
 - Fee-share minting leaves the vault temporarily overvalued
 - Price rounding effect becomes amplified for low prices
 - Repay does not distinguish interest from principal
 - Borrowing can consume funds needed for queued withdrawals
 - Excess repayment value can be captured by new deposits

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This report covers the security review for Tomorrow protocol, a permissioned asset management protocol.

Our security assessment was a full review of the code, spanning a total of 1 week.

During our review, we did not identify any major severity vulnerability.

We did identify some minor severity vulnerabilities and code optimizations.

All reported issues were fixed or acknowledged by the development team and subsequently verified by us.

We can confidently say that the overall security and code quality have increased after completion of our audit.

3. Security Review Details

- **Review Led by**

Kasper Zwijsen, Head of Audits

- **Scope**

The analyzed resources are located on:

<https://github.com/here-and-now-tech/tomorrow-contracts/tree/b95a1e1ed021575322f96bb748ca5398a582ef77>

The issues described in this report were fixed in the following commit:

<https://github.com/here-and-now-tech/tomorrow-contracts/tree/820aacf388833e13dd75b2586529495e9d640b0a>

- **Changelog**

■ 14 September 2026	Audit start
■ 22 September 2026	Initial report
■ 24 September 2026	Revision received
■ 2 October 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

- 1. Impact of the vulnerability
- 2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

▪ Severity Characteristics

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical	Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.
High	Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium

Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.

Low

Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.

Informational

Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

▪ Issue Symbolic Codes

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity

Number of findings

Critical	0
High	0
Medium	0
Low	4
Informational	3

Total:

7



Low

Informational



Fixed

Acknowledged

6. Weaknesses

This section contains the list of discovered weaknesses.

TMRW1-1 | Replacing the fee controller resets the high-water mark

Fixed 

Severity:

Low

Probability:

Rare

Impact:

Medium

Path:

TomorrowFeeController.sol, TomorrowVault.sol

Description:

Replacing the fee controller resets the high-water mark (HWM) to the current share price, instead of preserving the previous controller's HWM.

This can cause LPs to pay performance fees again when the vault recovers from a loss.

```
function _activate() internal {  
    // slither-disable-next-line timestamp -- nonzero is an activation sentinel; timestamp starts fee accrual only.  
    if (lastManagementCrystallization != 0) revert AlreadyActivated();  
    lastManagementCrystallization = block.timestamp;  
    highWaterMark = ITomorrowVault(vault).sharePrice();  
    emit FeeControllerActivated(highWaterMark);  
}
```

Root Cause

setFeeController() requires a fresh controller, whose _activate() sets:

```
highWaterMark = ITomorrowVault(vault).sharePrice();
```

Therefore, if the old HWM was 2.00 and the share price falls to 1.00, replacing the controller sets the new HWM to 1.00.

Remediation:

Preserve the outgoing controller's HWM when activating the replacement controller, rather than initializing it solely from the current share price.

TMRW1-4 | Management fees accrue before the first deposit

Fixed 

Severity:

Low

Probability:

Rare

Impact:

Low

Path:

```
./src/TomorrowFeeController.sol
```

Description:

The management fee clock starts when the vault is activated (or the last crystalize fee call), even if the vault has no deposits yet.

If the vault remains empty for a week, the first depositor is immediately diluted by paying a full week of management fees based on the amount he deposited.

Remediation:

In `_checkpointManagementFees`, check whether the total supply is zero. If so, update `lastManagementCrystallization` to `block.timestamp` and return early.

TMRW1-5 | Fee-share minting leaves the vault temporarily overvalued

Acknowledged

Severity:

Low

Probability:

Rare

Impact:

Low

Path:

```
./src/TomorrowVault.sol
```

Description:

When it crystallizes fees, it mints new shares to the treasury. This increases the total share supply, but the published **sharePrice()** does not change.

For example, if the vault publishes a price of **\$1.00** and then mints fee shares, the additional shares dilute the value of every share. The true value might now be **\$0.98**, while the vault still reports **\$1.00**.

Also the price cannot immediately be corrected. The oracle enforces **minimumPriceUpdateInterval**, so the next publication is rejected until the interval has passed.

This creates a period where the vault's share supply and published price are out of sync which affect deposits and withdrawals during this window. New deposits can receive fewer valuable shares than they paid for, while withdrawals can be accepted at the stale higher price, transferring value from remaining holders. The entire fee can effectively be avoided if the vault is withdrawn before the price is corrected.

TMRW1-7 | Price rounding effect becomes amplified for low prices

Fixed ✓

Severity:

Low

Probability:

Rare

Impact:

Medium

Path:

src/TomorrowPriceAdapter.sol:_scaleFrom#L102-L111

Description:

The adapter normalizes share prices to six decimals before the vault uses them to enforce the deposit cap and calculate minted shares. The relevant deposit flow is `_latest` -> `_scaleFrom` -> `TomorrowVault.deposit`, and it applies to any eligible depositor.

Normally, the normalized price should remain precise enough that a depositor receives ownership matching the assets they contributed. Later withdrawals should then pay that depositor only their fair share of any portfolio recovery.

However, `_scaleFrom` rounds down prices from sources with more than six decimals. An exact price of **0.000001999999** USDC is therefore normalized to **0.000001** USDC. Near this lower boundary, the error is almost 50%, even though less than one six-decimal price unit was discarded. `deposit` treats the rounded value as exact, so it can mint almost twice as many shares as it should and also understate the value counted toward the deposit cap.

The excess shares can later be redeemed at a higher normalized price through the regular withdrawal flow:

1. The vault has 1 trillion shares at a true price of **0.000001999999** USDC, but the adapter reports **0.000001** USDC.
2. An eligible user deposits \$100,000 and receives 100 billion shares instead of roughly 50 billion shares.
3. The portfolio recovers and the true price reaches **0.000002** USDC, which the adapter now reports as **0.000002** USDC.
4. The user requests redemption and, once processed through the normal FIFO queue, receives \$200,000. Most of the amount above their fair recovery comes from incumbent LPs.

```
function _scaleFrom(address source, uint256 value) internal view returns (uint256 normalized) {
    uint8 d = AggregatorV3Interface(source).decimals();
    if (d == 6) {
        normalized = value;
    } else if (d < 6) {
        normalized = value * 10 ** (6 - d);
    }
}
```

```
    } else {  
        normalized = value / 10 ** (d - 6);  
    }  
    if (normalized == 0) revert InvalidPrice();  
}
```

Remediation:

Keep the source's native precision, or use a higher common precision, throughout deposit, cap, and withdrawal accounting. If that is not practical, block deposits and new withdrawal acceptance when the relative normalization error or its aggregate value exceeds a conservative limit.

TMRW1-2 | Repay does not distinguish interest from principal

Acknowledged

Severity:

Informational

Probability:

Rare

Impact:

Informational

Path:

TomorrowVault.sol

Description:

`repay()` always reduces the borrower's **outstanding** balance:

```
config.outstanding -= uint128(Math.min(assets, config.outstanding));
```

This assumes that the full payment represents principal repayment. However, it does not track or distinguish between principal and interest payments.

As a result, if a borrower makes an interest payment through `repay()`, the payment is incorrectly deducted from `outstanding`, causing the recorded debt to become lower than the borrower's actual principal obligation.

For example:

- Borrowed: \$90k
- Funding cap: \$90k
- Borrower pays \$900/month interest through **repay()**
- After 12 payments, actual principal remains \$90k
- **outstanding** incorrectly falls to \$79.2k
- The vault therefore appears to have \$10.8k of available capacity

The borrower can then potentially draw additional funds while still owing the original \$90k principal.

Remediation:

Explicitly separate principal repayments from interest payments, e.g. provide a dedicated `payInterest()` function, and document how interest must be paid.

TMRW1-3 | Borrowing can consume funds needed for queued withdrawals

Acknowledged

Severity:

Informational

Probability:

Rare

Impact:

Informational

Path:

`./src/TomorrowVault.sol`

Description:

When a user requests a withdrawal, their shares are burned and the withdrawal is added to the queue. The user can no longer use those shares, but they are still waiting for the withdrawal to be accepted and the vault to provide the assets.

However, **fundBorrower()** does not take these pending withdrawals into account when lending out the vault's USDC.

Impact

An Operations Manager can lend out the vault's available USDC while withdrawals are already queued, potentially leaving the vault unable to process those withdrawals until additional liquidity becomes available.

Remediation:

Ensure **fundBorrower()** cannot reduce liquid assets below the amount required to satisfy pending withdrawal requests. Alternatively, borrowing could be paused while withdrawals are queued, but this is more restrictive than necessary.

TMRW1-6 | Excess repayment value can be captured by new deposits

Acknowledged

Severity:

Informational

Probability:

Rare

Impact:

Informational

Path:

`./src/tomorrowVault.sol`

Description:

When a borrower repays more than its outstanding principal, the excess remains (borrow interest repayment) in the vault as additional assets. However, the share price does not change until the next oracle publication.

Because the oracle enforces **minimumPriceUpdateInterval**, there can be a period where the vault already holds the additional repayment value but deposits still use the old price.

A new eligible depositor can therefore enter during this window and receive shares based on the old price, gaining part of the value that was earned before they deposited.

hexens x  Tomorrow

